

Architectural Specification for a Next-Generation Local-First Geospatial Hiking Application

Executive Summary of the Architectural Paradigm

The commercial geospatial application market is currently dominated by paradigms heavily reliant on centralized cloud infrastructure. In these traditional models, topographic basemaps, routing graphs, and user-generated trail data are stored on proprietary remote servers, necessitating recurring subscription fees to offset computational, storage, and bandwidth costs. This centralized architecture inherently restricts offline capabilities, degrades user data privacy, and introduces severe latency in geographically remote areas—the precise environments where hiking applications are utilized most frequently. Furthermore, commercial hiking platforms often gatekeep public domain data, repackaging openly accessible OpenStreetMap (OSM) and governmental geographic data behind prohibitive paywalls. The "Antigravity" paradigm represents a fundamental architectural inversion of this standard. It advocates for a one hundred percent "local-first," serverless deployment model characterized by zero server overhead, zero user subscription fees, and maximum client-side elegance. Leveraging recent advancements in browser APIs, WebAssembly (Wasm) compilation, and Cloud-Optimized geospatial data formats, it is now entirely feasible to construct a highly performant application utilizing standard web technologies, wrapped natively via lightweight frameworks such as Tauri or Capacitor.

Under this decentralized architecture, the client device assumes all responsibilities historically delegated to a backend server. Map rendering, spatial indexing, complex spatial queries, and turn-by-turn routing are executed entirely within the device's local memory sandbox. Basemaps are fetched from public static cloud buckets using precise byte-range requests and cached locally without the intervention of a dynamic tile server. Routing graphs are parsed directly within the browser using WebAssembly-compiled routing engines that operate independently of an internet connection. User data is synchronized exclusively through client-owned cloud providers via direct, serverless cryptographic integrations. The resulting system achieves total autonomy, operating indefinitely without a proprietary backend database, effectively neutralizing the overhead costs associated with traditional geospatial applications and returning sovereignty to the user.

Detailed Component Diagram

The following logical architecture outlines the rigid separation of concerns and the data flow within the client-side environment. This design prioritizes the isolation of the main user interface thread from computationally expensive geospatial operations.

Code snippet

```
graph TD
    subgraph Client_Application
        subgraph Main_Thread
            UI
            MapEngine
            ContourPlugin
            MapEngine <--> UI
            ContourPlugin --> MapEngine
        end
    end
```

```
    subgraph Web_Worker_Pool
        TileParser
        SpatialIndex
        RoutingEngine
        DataIngest
```

```
        DataIngest --> SpatialIndex
    end
```

```
    subgraph Storage_Layer
        OPFS
        IDB
    end
```

```
    Main_Thread <--> |SharedArrayBuffer / RPC| Web_Worker_Pool
    Web_Worker_Pool <--> |Synchronous Byte Reads| OPFS
    Web_Worker_Pool <--> |Asynchronous Metadata| IDB
end
```

```
subgraph External_Infrastructure
    OSM
    DEM
    UserCloud
end
```

```
DataIngest <--> |Rate-Limited Direct Query| OSM
TileParser <--> |HTTP 206 Partial Content| DEM
```

The architecture strictly enforces a boundary between the Main Thread and the Web Worker Pool. The Main Thread is reserved exclusively for Document Object Model (DOM) state management and WebGL rendering operations. All heavy computations—including parsing massive Cloud-Optimized GeoTIFFs or PMTiles, generating turn-by-turn routing via WebAssembly, executing Douglas-Peucker route smoothing, and evaluating spatial proximity queries—are relegated to parallel Web Workers. This ensures the application maintains a strict 60 frames-per-second (FPS) rendering cycle. Storage relies on a hybrid Origin Private File System (OPFS) and IndexedDB model, interfacing directly with external open-data infrastructure and user-sovereign storage providers.

1. Offline Tile Management & Render Engine

The presentation of complex topographic data demands a highly optimized rendering engine capable of handling dense vector geometries and high-resolution raster elevation models simultaneously. The selection of the rendering library and the underlying file format dictates the application's memory footprint, battery consumption, and the viability of completely offline operations.

Rendering Engine Comparison: MapLibre GL, Leaflet, and OpenLayers

The open-source geospatial JavaScript ecosystem provides three primary candidates for map rendering: Leaflet, OpenLayers, and MapLibre GL JS. For a high-performance, local-first application attempting to render vast networks of hiking trails on constrained mobile hardware, the underlying rendering technology—specifically the reliance on the DOM versus hardware-accelerated WebGL—is the critical differentiator.¹

Feature / Metric	Leaflet	OpenLayers	MapLibre GL JS
Rendering Pipeline	DOM (HTML/SVG) + Canvas	Canvas + WebGL (Optional)	Native WebGL
Vector Tile Support	Poor (Requires plugins, heavy DOM)	Good (Complex configuration)	First-Class (Native WebGL rendering)
Library Bundle Size	Extremely lightweight (~40KB minified)	Heavy (Feature-rich GIS toolkit)	Moderate (Optimized for vector pipelines)
Large Dataset Performance	Degrades exponentially > 1,000 features	Scales moderately, consistent performance	Excels with > 50,000 features

3D Terrain & Pitch	Not natively supported	Limited to external plugins	First-class 3D terrain and sky layers
-------------------------------	------------------------	-----------------------------	---------------------------------------

Leaflet is widely recognized for its minimal bundle size, zero dependencies, and extreme simplicity.² However, Leaflet's architecture is fundamentally built around the older paradigm of raster slippy maps. When rendering vector geometries (such as GPS tracks or trail networks), Leaflet converts these data points into SVG or HTML elements injected directly into the DOM.² While this provides excellent accessibility out of the box, attempting to render a hiking network comprising hundreds of thousands of vertices results in catastrophic memory bloat and UI thread blocking.¹

OpenLayers provides a highly comprehensive suite of geographic information system (GIS) tools, robust projection support, and advanced spatial analysis capabilities.² However, its enterprise-grade architecture results in a heavy memory footprint and a steep learning curve. While OpenLayers handles large datasets far better than Leaflet due to its Canvas rendering capabilities, it is generally over-engineered for a consumer-facing mobile application focused primarily on rendering and routing.¹

MapLibre GL JS emerges as the definitive, architecturally superior choice. Originally a community-driven fork of Mapbox GL JS, MapLibre relies exclusively on WebGL to render vector tiles directly via the device's Graphics Processing Unit (GPU).⁴ Rather than creating expensive DOM nodes, MapLibre translates geographic coordinates into WebGL vertex buffers. While MapLibre exhibits slightly slower initialization times for negligible datasets, its rendering performance scales linearly, significantly outperforming Leaflet when rendering dense trail geometries exceeding 50,000 polygons or multi-vertex linestrings.¹ Furthermore, MapLibre natively supports 3D terrain rendering, smooth sub-pixel zooming, and camera pitch rotation, which are absolute prerequisites for a modern topographic hiking application.⁴

PMTiles and Cloud-Optimized Vector Storage

Traditional geospatial applications utilize active map tile servers (e.g., heavily cached PostgreSQL/PostGIS databases sitting behind a dynamic tile server like Martin, pg_tileserv, or Tegola) to serve Z/X/Y vector coordinates to the client. This deeply violates the serverless requirement of the antigravity paradigm. The architectural solution is PMTiles, a single-file, cloud-optimized archive format designed for hosting tilesets without a server.⁹

PMTiles utilizes an internal directory structure based on a packed Hilbert curve index. The client application utilizes the @protomaps/pmtiles JavaScript library, integrating directly into MapLibre via the addProtocol method.⁶

```

JavaScript
import Protocol from "pmtiles"
import maplibregl from "maplibre-gl"

const protocol = new Protocol()
maplibregl.addProtocol("pmtiles", protocol.tile)

const map = new maplibregl.Map({
  container: 'map',
  style:
    version 8
    sources
      "protomaps"
      "type" "vector"
      "url" "pmtiles://https://example.com/trails.pmtiles"
    layers

```

When the user pans or zooms the map, MapLibre calculates the necessary Z/X/Y tiles required for the viewport. The PMTiles protocol intercepts this request, queries the PMTiles archive header (typically the first 512 bytes) via an HTTP Range request to locate the specific byte offsets of the required tiles, and then executes a subsequent HTTP 206 Partial Content request for only those targeted data chunks.⁶ This allows the application to read directly from a massive, multi-gigabyte global topographic dataset hosted on a static public Amazon S3 bucket, consuming only kilobytes of bandwidth per view, completely bypassing the need for a dedicated tile API.⁶

For true offline functionality, these .pmtiles archives can be downloaded directly to the device's local storage. A plugin such as maplibre-offline-pmtiles facilitates the local storage and retrieval of these archives, parsing both Mapbox Vector Tile (MVT) and MapLibre Tile (MLT) formats directly from the local file system without any network activity.¹¹

Client-Side Tile Caching Boundaries: OPFS vs. IndexedDB

The caching of multi-gigabyte PMTiles archives, Digital Elevation Models, and local vector datasets requires careful navigation of browser storage quotas and performance bottlenecks.

Modern web applications generally rely on the Cache API or IndexedDB. However, IndexedDB incurs severe performance overhead when handling massive ArrayBuffer objects.¹² Storing a 1GB offline region in IndexedDB requires the browser's structured clone algorithm to serialize and deserialize the data during read/write operations, leading to massive memory spikes, transaction lifecycle costs, and main-thread blocking.¹²

The Origin Private File System (OPFS) provides a private, sandboxed filesystem optimized for high-performance read and write access to raw binary data. Unlike the File System Access API, OPFS does not trigger intrusive user permission dialogs and operates directly and invisibly within the browser's origin sandbox.¹²

The optimal architecture implements a dual-tier storage strategy to maximize efficiency¹²:

1. **Metadata Layer (IndexedDB):** Utilized exclusively to store small, structured JSON metadata regarding the downloaded map regions, bounding boxes, expiration dates, synchronization states, and user preferences.¹²
2. **Binary Layer (OPFS):** Reserved strictly for storing the raw binary .pmtiles archives, .mbtiles, or Digital Elevation Model (DEM) .png/.webp files.¹²

OPFS allows highly optimized, synchronous, in-place reads when executed from within Web Workers via the `createSyncAccessHandle()` method. This is a crucial architectural distinction. To query a file size or read a specific chunk using the standard Cache API, the entire file must often be loaded into memory or processed asynchronously.¹² In OPFS, retrieving `file.size` is instantaneous, and the application can execute partial read operations at arbitrary byte offsets synchronously.¹² This maps perfectly to the byte-range requirements of the PMTiles format, allowing a background Web Worker to act as a high-speed, local virtual tile server reading directly from the device's solid-state drive.¹²

Dynamic Contour Rendering and Elevation Decoding

A core feature of any topographic hiking map is the visualization of elevation contours. Pre-generating vector contour tiles for global landmasses is computationally expensive and yields unmanageably large file sizes. The serverless paradigm mandates client-side generation. Using the open-source `maplibre-contour` plugin, the application can generate vector contour isolines dynamically within a Web Worker from standard RGB Raster DEM tiles.⁹ The plugin fetches raster elevation tiles, decodes the RGB color values of the pixels into raw heights in meters, generates vector isolines using a highly optimized marching-squares algorithm, and encodes the output as a Mapbox Vector Tile (MVT) buffer.¹⁵ This dynamically generated vector tile is then fed back to MapLibre as a standard vector source, allowing the contours to be styled seamlessly with standard MapLibre syntax.¹⁴

It is vital to distinguish between elevation encodings. Open data sources generally use the "Terrarium" encoding format (originating from Mapzen), while Mapbox uses a proprietary RGB encoding.¹⁸ If the encoding property is misconfigured, the application will silently produce highly inaccurate elevation topologies, placing contour lines at completely incorrect altitudes.¹⁸ The mathematical decoding functions implemented by the Web Worker differ significantly: For **Mapbox** encoded tiles, the elevation in meters is calculated as:

$$elevation = -10000 + \frac{(R \times 256 \times 256 + G \times 256 + B)}{10}$$

For **Terrarium** encoded tiles, the elevation is calculated as:

$$elevation = (R \times 256 + G + \frac{B}{256}) - 32768$$

By correctly identifying the source format, the client application can process high-resolution elevation data globally without maintaining massive pre-rendered contour vector datasets.¹⁸

2. Open Data Ingestion Pipelines (Serverless)

Without a centralized backend database to aggregate, clean, format, and distribute trail data, the client application must ingest geospatial data directly from public infrastructure. This ingestion pipeline must be highly resilient, bandwidth-efficient, strictly respectful of public API rate limits, and capable of serializing data into performant local formats.

Overpass API Query Optimization

The OpenStreetMap (OSM) database represents the most comprehensive global repository of un-gated trail data. The Overpass API acts as a read-only database engine over the web, allowing the client to execute highly specific queries against the OSM dataset.²¹ In OSM topology, physical trails are usually mapped as way objects with specific tags such as `highway=path`, `highway=footway`, or `route=hiking`.²² Long-distance named trails (e.g., the Appalachian Trail) are mapped as relation objects, which group multiple discrete ways together into a cohesive route.²²

Querying Overpass directly from a client application introduces severe rate-limiting constraints. The public Overpass API utilizes a strict slot-based load shedding algorithm to manage server resources. A single IPv4 address is typically allocated only two concurrent slots, and users are strongly expected to limit their activity to fewer than 10,000 queries and less than 1GB of payload data per day.²⁵ Exceeding these limits forces the query into a queue, eventually triggering HTTP 429 (Too Many Requests) or HTTP 504 (Gateway Timeout) responses.²⁶ Furthermore, Overpass actively blocks requests originating from default browser user agents or generic scripts, frequently returning HTTP 406 (Not Acceptable) errors if a unique identification header is missing, preventing automated scraping.²⁷

To query the OSM database without disruption, the application must implement the following optimization strategies:

1. **Custom User-Agent Enforcement:** All fetch requests must include a strictly defined User-Agent header (e.g., `X-Client-Id: Antigravity-Hiking-App/1.0`) to comply with fair-use policies and avoid automated 406 blocks.²⁷
2. **Geographical Bounding Box Optimization:** Overpass Query Language (QL) requests

must be tightly constrained to the user's immediate vicinity. A query fetching hiking routes should restrict its search to the current viewport bounding box using the (`bbox`) parameter to prevent massive, cross-continental data dumps.²⁹

3. **Precise Data Filtering:** Extracting only specific nodes, ways, and relations prevents bandwidth bloat. A highly optimized Overpass QL query limits the payload to hiking routes and includes necessary geometry without pulling superfluous metadata.

Code snippet

```
[
  way[route=hiking]({bbox});
  way[route=hiking]({bbox});
  way[route=hiking]({bbox});
  way[route=hiking]({bbox});
  way[route=hiking]({bbox});
]
```

4. **Local Caching & Spatial Verification:** Once successfully queried, the JSON payload must be immediately serialized and stored in OPFS. Before initiating a network request to Overpass, a local spatial index must verify if the current bounding box intersects with previously cached regions. If it does, the client serves the data locally, bypassing the network entirely.

Zero-Cost Static Endpoints

Relying exclusively on the live, dynamically generated Overpass API is a substantial risk for a consumer application facing unpredictable traffic spikes. The architecture must heavily leverage secondary static datasets stored on open-access object storage to guarantee reliability.

Waymarked Trails Dataset: For established long-distance hiking routes, the Waymarked Trails initiative provides regularly updated, pre-compiled static files (GPX, KML) derived directly from OSM route relations.²⁴ The client application can fetch these static resources directly, bypassing the need to execute complex relation-assembly queries against Overpass, significantly reducing parsing time and bandwidth.²⁴

Digital Elevation Models (DEM): High-resolution global terrain data is required for 3D mapping and elevation profiling. The Copernicus DEM (GLO-30) dataset, which offers a 30-meter global resolution, is freely available and hosted on the AWS Registry of Open Data as Cloud Optimized GeoTIFFs (COGs).³⁴ For localized data within the United States, the United States Geological Survey (USGS) provides the National Map 3D Elevation Program (3DEP) and historical topographic tile cache services. These are heavily subsidized and accessible via public ArcGIS REST services, WMTS, and standard XYZ tile formats (<http://basemap.nationalmap.gov/ArcGIS/rest/services/USGSTopo/MapServer/tile/{z}/{y}/{x}>) without API keys.³⁶

Standardized Storage Formats: FlatGeobuf vs. GeoJSON

Ingesting raw GeoJSON payloads from the Overpass API is computationally disastrous for a mobile client. GeoJSON is a bloated, text-based format that requires the browser to parse the entire file into memory (via `JSON.parse`) before any rendering or spatial queries can occur. For extensive regional trail networks, this operation blocks the main thread, spikes garbage collection, and routinely exceeds the memory capacity of mid-range mobile devices.⁴⁰

The application must convert ingested GeoJSON into **FlatGeobuf** for local storage.

FlatGeobuf is a highly performant, binary encoding format for geographic data based on FlatBuffers.⁴² Its defining architectural characteristic is the inclusion of a packed Hilbert R-Tree spatial index embedded directly within the file's header.⁴¹

The FlatGeobuf data schema operates through a strict binary layout:

1. **Magic Bytes (8 bytes):** Identifies the file format and specification version.
2. **Header (Variable length):** Contains dataset metadata, coordinate reference systems (CRS), and column definitions.
3. **Spatial Index (Optional, Static size):** A packed Hilbert R-Tree mapping bounding boxes to byte offsets in the data block.
4. **Data Block:** The actual geometric features, serialized as variable-size FlatBuffers.⁴²

When the application needs to render a localized 5-mile area from a massive 12GB national trail FlatGeobuf file stored in OPFS, it does not read the entire file. Instead, the Web Worker reads only the compact R-Tree index at the beginning of the file via an HTTP Range request (or precise OPFS `read()` offsets). It identifies the specific byte ranges corresponding to the user's current bounding box, and subsequently executes read requests for only those targeted bytes containing the intersecting features.⁴¹ This "Cloud-Native Vector" approach ensures that even globally comprehensive datasets can be queried in milliseconds, with a near-zero memory footprint.⁴³

3. Client-Side Computation & Performance

A true serverless hiking application must perform heavy geospatial computations locally. Without offloading these intensive mathematical tasks to optimized background threads, the user interface will suffer severe frame drops, resulting in an unresponsive application.

Web Worker Architecture: Parsing, Smoothing, and Profiling

To achieve the antigravity vision, the architecture implements a strict actor-model threading structure. The main browser thread's sole purpose is UI manipulation and MapLibre GL instance management. Background Web Workers handle all data manipulation.

Parsing and Serialization: When the user imports a large GPX file representing a recorded track, parsing the XML structure on the main thread is prohibitively slow. A dedicated Web Worker ingests the text stream, parses the XML nodes into geometric coordinates, and serializes the result into a binary format.¹⁰

Route Smoothing (Douglas-Peucker Algorithm): Raw GPS tracks often contain excessive, noisy vertices that contribute nothing to visual fidelity but massively inflate rendering costs. Within the Web Worker, the application applies the Ramer-Douglas-Peucker algorithm. This mathematical process recursively reduces the number of points in a curve that is approximated

by a series of points, eliminating redundant vertices that fall within a defined tolerance distance. This dramatically reduces the memory required to pass the geometry to the MapLibre renderer.

Elevation Profiling: To generate an elevation chart for a planned route, the Web Worker samples the route's coordinates against locally cached DEM raster tiles. The worker decodes the Terrarium PNG into a Float32Array of elevation values, interpolates the route's path across the pixel grid, and extracts the altitude for each vertex, passing an array of distances and elevations back to the UI for charting.¹⁵

Communication between the main thread and the Web Workers must strictly utilize Transferable objects (such as ArrayBuffer). Utilizing standard postMessage with JSON payloads triggers the browser's structured cloning algorithm, creating a deep, synchronous copy of the data, which immediately doubles the memory allocation and spikes CPU usage. By transferring ownership of the ArrayBuffer, data is passed seamlessly between threads with zero-copy overhead.¹²

Client-Side Spatial Indexing

A common requirement in hiking applications is the "trails near me" feature. Executing a point-in-polygon or proximity search across millions of coordinate pairs using linear scans is mathematically unfeasible and will lock the application thread. The client must maintain a highly optimized, resident spatial index.

While RBush is a popular JavaScript R-tree implementation for dynamic bounding box searches, it is sub-optimal for static, pre-compiled datasets.⁴⁵ Because RBush allows nodes to be added and removed dynamically, the resulting tree can become fragmented, leading to slower search times.⁴⁵

For the local-first application, **Flatbush** (and its point-specific variant, KDBush) is the architecturally superior choice.⁴⁶ Flatbush generates a static, packed, ABI-stable Hilbert R-Tree.⁴⁴ Unlike RBush, Flatbush requires the index to be built completely upfront; geometries cannot be appended later.⁴⁴ Because the tree is static and packed, all nodes are at full capacity, meaning it does not waste memory reserving empty node space for future additions.⁴⁴

Crucially, the entire Flatbush index is structured as a contiguous, single ArrayBuffer with a well-defined, stable memory layout.⁴⁴ This is profoundly advantageous for a Web Worker architecture. The index can be built once by a background worker, saved natively as a compact binary file to OPFS, and subsequently passed to the main thread (or other workers) as a zero-copy transferable object.⁴⁴ This allows the application to perform lightning-fast nearest-neighbor and bounding box queries on millions of trail nodes within milliseconds, utilizing minimal heap memory.⁴⁰

WebAssembly (Wasm) Offline Routing Engines

The provision of offline, turn-by-turn routing is the most computationally demanding feature of the application. Commercial maps utilize robust server-side APIs running instances of advanced routing algorithms like the Open Source Routing Machine (OSRM), GraphHopper, or

Valhalla.⁵¹

Attempting to run OSRM entirely within a client device's browser is highly problematic. OSRM utilizes Contraction Hierarchies (CH) as its primary routing algorithm.⁵¹ CH achieves exceptionally fast point-to-point query times by pre-processing the road network graph and precomputing thousands of routing shortcuts.⁵¹ However, this fully pre-processed graph must be loaded entirely into RAM. Depending on the geographical dataset, OSRM requires massive, monolithic memory allocations that far exceed browser WebAssembly constraints.⁵⁴ Furthermore, because the routing metrics are baked directly into the graph during pre-processing, OSRM cannot dynamically alter routing behavior (e.g., dynamically penalizing steep elevation gains, or strictly avoiding unpaved surfaces) at runtime without rebuilding the entire graph.⁵⁴

Alternatively, GraphHopper provides robust Java-based routing. Through projects utilizing TeaVM, GraphHopper can be compiled from Java bytecode directly to JavaScript, allowing it to run in the browser without a backend.⁵⁵ However, JavaScript-emulated routing engines suffer from severe performance bottlenecks compared to native execution, and managing the partial loading of massive graph files into browser memory remains a complex challenge for TeaVM implementations.⁵⁵ Similarly, BRouter offers a pure JavaScript routing engine tailored explicitly for cycling and hiking, utilizing configurable .brf text profiles.⁵⁶ While effective for specific niches, its purely interpreted execution lacks the raw performance required for instantaneous, continent-scale matrix calculations on mobile devices.⁵⁶

Valhalla emerges as the optimal routing engine for the antigravity paradigm.²⁸ Unlike OSRM, Valhalla does not attempt to store the entire graph in monolithic memory; instead, it relies on a highly flexible, tiled data structure.⁵⁴ Valhalla organizes routing data into three hierarchical levels: Highway (Level 0, 4° bounding box size), Arterial (Level 1, 1° size), and Local (Level 2, 0.25° size).⁵⁹ During route computation, Valhalla's Thor pathfinding engine traverses these hierarchical tiles, loading only the necessary geographic bounds from the file system into memory at any given time, making it uniquely suited for memory-constrained mobile devices.²⁸ Furthermore, Valhalla's Sif component performs dynamic costing at runtime rather than relying on pre-baked shortcuts.²⁸ Valhalla calculates the impedance of an edge on the fly based on a rich set of attributes embedded in the tiles (e.g., elevation changes, road surface types, bicycle vs. pedestrian suitability). This allows the application to generate highly customized hiking profiles, avoiding high-traffic roads or prioritizing scenic routes, without requiring a separate pre-processed graph for every transit mode.⁵⁴

Through projects like @jansoft/mbujkanji-valhalla-wasm, Valhalla is successfully cross-compiled from C++ to WebAssembly.⁶² Wasm allows the routing engine to execute within the browser Sandbox at near-native speeds. The application can load regional .tar routing tiles dynamically via Valhalla's API, pass origin and destination coordinates, and receive detailed turn-by-turn navigational maneuvers entirely offline.⁶²

TypeScript

```
import { createRouter } from '@jansoft/mbujkanji-valhalla-wasm'
```

```
const router = createRouter()
```

```
await router.init()
```

```
// Load regional Valhalla tiles from OPFS into the Wasm memory space
```

```
const tilesBuffer = await readTilesFromOPFS('region.tar')
```

```
await router.loadTiles(tilesBuffer)
```

```
const result = await router.route()
```

```
locations
```

```
costing: 'pedestrian' // Dynamic Sif costing for hiking
```

```
directions_type: 'maneuvers'
```

```
console.log(result.trip)
```

```
router.dispose() // Free Wasm memory
```

Engine	Algorithm Approach	Memory Footprint	Runtime Flexibility	Browser / Wasm Feasibility
OSRM	Contraction Hierarchies	Very High (Monolithic)	Low (Baked-in costs)	Poor (Exceeds Wasm memory)
GraphHopper	A* / Dijkstra / CH	High	Moderate	Fair (via TeaVM JS emulation)
BRouter	A* with Elevation	Low	High (Custom.brf profiles)	Good (Pure JS)
Valhalla	Tiled A*	Low (Tile-based)	Very High (Dynamic)	Excellent (Native Wasm)

	(Thor/Sif)	loading)	costing)	cross-compilation)
--	------------	----------	----------	--------------------

4. User Data Sovereignty & Sync

Eliminating the central server database inherently raises a critical data management question: how is user-generated data—such as recorded GPS tracks, saved waypoints, and custom map layers—backed up and synchronized seamlessly across multiple devices? The antigravity architecture guarantees strict data sovereignty by utilizing the user's existing, authenticated cloud storage services rather than a proprietary backend.

Zero-Server Sync Architecture

Instead of routing user data through a developer-maintained database, the application interfaces directly with provider APIs (e.g., Google Drive API, Dropbox API) originating strictly from the client device.⁶⁴ The application logic utilizes native Web APIs to serialize user trails into standard GPX or FlatGeobuf files and uploads them directly to an application-specific folder within the user's cloud drive (e.g., /Apps/AntigravityApp/). This ensures that if the application ceases development, the user retains absolute ownership and access to their data.

Client-Side OAuth 2.0 PKCE Authorization

Integrating with third-party cloud APIs strictly from a client-side application introduces unique, severe security challenges. Historically, Single-Page Applications (SPAs) relied on the OAuth 2.0 Implicit Grant Flow, which returns the access token directly in the URL redirect URI.⁶⁴ This exposes the highly sensitive token to malicious interception via cross-site scripting (XSS) or rogue browser extension vulnerabilities.⁶⁴ Furthermore, client-side applications cannot safely store a static client_secret (a fundamental requirement for standard OAuth), as the application's source code is entirely exposed to the user.⁶⁵

The architecture must implement the **Authorization Code Flow with Proof Key for Code Exchange (PKCE)**.⁶⁴ PKCE was explicitly designed by the IETF to secure public clients executing without a backend. The cryptographic exchange operates as follows:

1. The client application dynamically generates a high-entropy random string called the `code_verifier`.
2. The client hashes the verifier using SHA-256 and base64url-encodes it to create the `code_challenge`.
3. The client directs the user to the provider's authorization endpoint (e.g., Google or Dropbox), passing the `code_challenge` in the URL parameters.
4. Upon successful user login, the provider returns an authorization code to the client.
5. The client requests an access token by sending the authorization code *alongside the original, unhashed code_verifier* directly to the provider's token endpoint.
6. The provider hashes the received `code_verifier` and compares it to the initially stored

code_challenge. If the hashes match, the token is securely issued.⁶⁶ Because the code_verifier is dynamically generated per session and never exposed in the initial redirect, a malicious actor intercepting the authorization code cannot exchange it for an access token without possessing the original verifier.⁶⁶ This allows the hiking application to securely authenticate against Dropbox and Google Drive entirely client-side, retrieving tokens without exposing hardcoded application secrets.⁶⁵

OS Level Constraints: iOS, iCloud, and Capacitor

While Dropbox and Google Drive offer robust REST APIs for direct client integration, integrating natively with Apple's iCloud via iOS poses significant architectural challenges. When deploying the web application to iOS via a native wrapper like Capacitor or Tauri, developers frequently attempt to utilize iCloud Drive directly for file synchronization.⁶⁸

However, Apple's iOS strictly manages iCloud Drive synchronization at the system level to aggressively prioritize battery life and thermal management.⁶⁹ Applications cannot programmatically force an iCloud synchronization event.⁶⁹ If a user records a grueling 10-hour hiking track, saves it to the local iCloud Drive directory within the Files app, and terminates the application, the operating system determines entirely arbitrarily when to push that file to the cloud. This results in severe data consistency issues if the user attempts to view the route on another device immediately.⁶⁹

For robust local metadata that must persist across installations and backups, the application utilizes the Capacitor Preferences API. However, this is backed by UserDefaults on iOS, which is suitable only for minimal key-value pairs (e.g., user settings), not massive geospatial datasets.⁶⁸ To ensure large file synchronization works reliably across all ecosystems without unpredictable system-level delays, the application should prioritize the direct, API-driven PKCE OAuth flows to Google Drive and Dropbox over native iCloud Drive background synchronization.⁶⁵

Risk Matrix & Mitigation Strategies

Implementing a pure client-side geospatial architecture exposes the application to severe bottlenecks primarily related to browser sandboxing limitations, memory constraints, and the monopolistic practices of mobile operating systems.

Identified Risk	Impact Level	Mechanism & Mitigation Strategy
Browser Storage Eviction (iOS Safari)	Critical	Apple's WebKit engine severely limits Progressive Web Apps (PWAs) to protect App Store revenues. ⁷¹ Safari aggressively clears IndexedDB and OPFS data

		via a "least-recently-used" eviction policy when device storage is under pressure.⁷³ To mitigate this, the application must prompt the user to enable the Persistent Storage API.⁷³ Furthermore, deploying the application via native web-views (Tauri or Capacitor) shields the storage from Safari's standard web browser eviction algorithms, granting permanent filesystem access and bypassing Apple's artificial PWA limitations.⁶⁸
WebAssembly 4GB Memory Constraints	High	Wasm environments within browsers (specifically V8 engine implementations) are currently restricted to a maximum 4GB memory heap due to 32-bit pointer constraints (wasm32).⁷⁴ Running Valhalla matrix algorithms or loading excessive routing tile layers can spike memory, resulting in a fatal RangeError: WebAssembly.Memory() crash.⁷⁵ Mitigation involves aggressive garbage collection within the Web Worker, restricting Valhalla route requests to localized subsets, and ensuring the mjolnir.max_cache_size configuration is strictly limited to prevent tile

		caching from exceeding Wasm memory bounds.⁷⁵
Main Thread Blocking & UI Jitter	High	Heavy geospatial computation (parsing XML, building Flatbush indexes, routing) blocks the UI thread, making map panning stutter and rendering the app unresponsive. Mitigation requires all MapLibre interactions, Flatbush indexing, and Valhalla routing to execute exclusively within Web Workers. SharedArrayBuffer must be used to transfer data between threads to achieve zero-copy memory transfers, preventing CPU spikes caused by standard structured cloning serialization.⁴⁴
Overpass API IP Blacklisting	Medium	Requesting bulk data triggers the slot-based rate limiter, leading to HTTP 429, 504, or 406 errors.²¹ Mitigation requires implementing exponential backoff algorithms, strict geographical bounding box limits ({{{bbox}}}), and ensuring every network request carries a unique, non-generic User-Agent to comply with Overpass usage guidelines.²⁷ Integrating static FlatGeobuf files as a primary data source heavily

		reduces reliance on the live API. ⁴³
PWA Geofencing & Background Limits	Medium	Apple severely restricts PWAs on iOS. Geofencing and background location tracking are entirely unsupported by WebKit, rendering browser-based track recording impossible when the screen is locked. ⁷¹ The application must be bundled via Capacitor or Tauri for deployment to mobile app stores to request native background location permissions, enabling uninterrupted track recording while the device is asleep. ⁷¹

The transition from a centralized server architecture to a distributed, local-first paradigm represents a significant leap forward in the resilience and accessibility of geospatial applications. By leveraging Cloud-Optimized PMTiles and FlatGeobuf formats alongside the Origin Private File System, the application achieves seamless rendering and data retrieval without backend APIs. Integrating Flatbush R-Trees and Valhalla WebAssembly enables powerful proximity querying and dynamic routing within the strict constraints of the browser sandbox. Finally, executing client-side OAuth PKCE flows ensures absolute user data sovereignty, eliminating the requirement for proprietary storage. This architectural specification successfully realizes the "Antigravity" vision, delivering a high-performance, subscription-free, and relentlessly autonomous hiking application.

Works cited

1. Vector Data Rendering Performance Analysis of Open-Source Web Mapping Libraries, accessed June 11, 2026, <https://www.mdpi.com/2220-9964/14/9/336>
2. Map libraries popularity: Leaflet vs MapLibre GL vs OpenLayers - Geoapify, accessed June 11, 2026, <https://www.geoapify.com/map-libraries-comparison-leaflet-vs-maplibre-gl-vs-openlayers-trends-and-statistics/>
3. Week 16: Web mapping: Leaflet vs MapLibre vs OpenLayers — LaunchDetect Academy, accessed June 11, 2026, <https://launchdetect.com/academy/week/16/>
4. What is the difference between these mapping libraries - Stack Overflow, accessed June 11, 2026,

- <https://stackoverflow.com/questions/79909996/what-is-the-difference-between-these-mapping-libraries>
5. Mapping libraries: a practical comparison - GIS CARTA, accessed June 11, 2026, <https://giscarta.com/blog/mapping-libraries-a-practical-comparison>
 6. Offline Maps with Protomaps in Maplibre - blog, accessed June 11, 2026, <https://blog.wxm.be/2024/01/14/offline-map-with-protomaps-maplibre.html>
 7. Overview - MapLibre GL JS, accessed June 11, 2026, <https://maplibre.org/maplibre-gl-js/docs/examples/>
 8. 3D Terrain - MapLibre GL JS, accessed June 11, 2026, <https://maplibre.org/maplibre-gl-js/docs/examples/3d-terrain/>
 9. Plugins - MapLibre GL JS, accessed June 11, 2026, <https://maplibre.org/maplibre-gl-js/docs/plugins/>
 10. PMTiles for MapLibre GL - Protomaps Docs, accessed June 11, 2026, <https://docs.protomaps.com/pmtiles/maplibre>
 11. A plugin for MapLibre GL JS to manage offline maps in PMTiles format. - GitHub, accessed June 11, 2026, <https://github.com/makinacorp/maplibre-offline-pmtiles>
 12. 3x faster project loads with the origin private file system, accessed June 11, 2026, <https://barndoor.lumafield.com/3x-faster-project-loads-with-the-origin-private-file-system/>
 13. Storage for the web | Articles, accessed June 11, 2026, <https://web.dev/articles/storage-for-the-web>
 14. Add Contour Lines. | JavaScript maps SDK - MapTiler documentation, accessed June 11, 2026, <https://docs.maptiler.com/sdk-js/examples/contour-lines/>
 15. onthegomap/maplibre-contour: Render contour lines from ... - GitHub, accessed June 11, 2026, <https://github.com/onthegomap/maplibre-contour>
 16. Design Proposal: Contour Line Source from Raster DEM Tiles #583 - GitHub, accessed June 11, 2026, <https://github.com/maplibre/maplibre-style-spec/issues/583>
 17. Add Contour Lines - MapLibre GL JS, accessed June 11, 2026, <https://maplibre.org/maplibre-gl-js/docs/examples/add-contour-lines/>
 18. [Skill] maplibre-terrain-patterns — Terrain, hillshade, and DEM sources · Issue #19 - GitHub, accessed June 11, 2026, <https://github.com/maplibre/maplibre-agent-skills/issues/19>
 19. Mapbox Terrain-RGB v1 | Tilesets | Mapbox Docs, accessed June 11, 2026, <https://docs.mapbox.com/data/tilesets/reference/mapbox-terrain-rgb-v1/>
 20. Optimization of RGB DEM tiles for dynamic hill shading with Mapbox GL or MapLibre GL, accessed June 11, 2026, <https://medium.com/@frederic.rodrigo/optimization-of-rgb-dem-tiles-for-dynamic-hill-shading-with-mapbox-gl-or-maplibre-gl-55bef8eb3d86>
 21. Overpass API - OpenStreetMap Wiki, accessed June 11, 2026, https://wiki.openstreetmap.org/wiki/Overpass_API
 22. What is an overpass query to get OSM hiking trails in the US? [closed] - GIS StackExchange, accessed June 11, 2026, <https://gis.stackexchange.com/questions/485420/what-is-an-overpass-query-to->

[get-osm-hiking-trails-in-the-us](#)

23. Overpass QL Intro. OpenStreetMap is probably something I... | by Pavel Saman - Medium, accessed June 11, 2026, <https://samanpavel.medium.com/overpass-ql-intro-e9be27a6e7b6>
24. Rendering OSM data - Waymarked Trails - Hiking, accessed June 11, 2026, <https://hiking.waymarkedtrails.org/#help-about>
25. Rate Limiting for Overpass-API Requests · Issue #605 · maproulette/maproulette3 - GitHub, accessed June 11, 2026, <https://github.com/maproulette/maproulette3/issues/605>
26. Commons - Overpass API, accessed June 11, 2026, <https://dev.overpass-api.de/overpass-doc/en/preface/commons.html>
27. Overpass API performance issues - Page 5 - Help and support, accessed June 11, 2026, <https://community.openstreetmap.org/t/overpass-api-performance-issues/140598?page=5>
28. GitHub - valhalla/valhalla: Open Source Routing Engine for OpenStreetMap, accessed June 11, 2026, <https://github.com/valhalla/valhalla>
29. Querying highways with Overpass QL for calculating routes - GIS StackExchange, accessed June 11, 2026, <https://gis.stackexchange.com/questions/272084/querying-highways-with-overpass-ql-for-calculating-routes>
30. Query-to-Map: All paths with Hiking Route relationship : r/openstreetmap - Reddit, accessed June 11, 2026, https://www.reddit.com/r/openstreetmap/comments/s97ajb/querytomap_all_paths_with_hiking_route/
31. Waymarked Trails - OpenStreetMap Wiki, accessed June 11, 2026, https://wiki.openstreetmap.org/wiki/Waymarked_Trails
32. Waymarked Trails - The best free trail data on the web - Explores Inc., accessed June 11, 2026, <https://exploresinc.com/2021/02/25/waymarked-trails-the-best-free-trail-data-on-the-web/>
33. Downloading hiking trail gpx from OpenStreetMap - YouTube, accessed June 11, 2026, <https://www.youtube.com/watch?v=yufWqvImOdo>
34. Copernicus GLO-30 Digital Elevation Model - OpenTopography, accessed June 11, 2026, <https://portal.opentopography.org/raster?opentopoID=OTSDEM.032021.4326.3>
35. Copernicus Digital Elevation Model (DEM) - Registry of Open Data on AWS, accessed June 11, 2026, <https://registry.opendata.aws/copernicus-dem/>
36. topoView | USGS - National Geologic Map Database, accessed June 11, 2026, <https://ngmdb.usgs.gov/topoview/>
37. USGSTopo (MapServer) - The National Map, accessed June 11, 2026, <https://basemap.nationalmap.gov/arcgis/rest/services/USGSTopo/MapServer>
38. What are the base map services (or URLs) used in The National Map? - USGS.gov, accessed June 11, 2026, <https://www.usgs.gov/faqs/what-are-base-map-services-or-urls-used-national->

[map](#)

39. USA: Topographic Maps – USGS Topo (WMS/WMTS/Tile Server/ArcGIS), accessed June 11, 2026, <https://support.plexearth.com/hc/en-us/articles/4404076245777-USA-Topographic-Maps-USGS-Topo-WMS-WMTS-Tile-Server-ArcGIS>
40. A dive into spatial search algorithms | by Vladimir Agafonkin - maps for developers, accessed June 11, 2026, <https://blog.mapbox.com/a-dive-into-spatial-search-algorithms-ebd0c5e39d2a>
41. FlatGeobuf in JavaScript - Cloud-Optimized Geospatial Formats Guide, accessed June 11, 2026, <https://guide.cloudnativegeo.org/flatgeobuf/flatgeobuf-in-js.html>
42. FlatGeobuf | flatgeobuf, accessed June 11, 2026, <http://flatgeobuf.org/>
43. Filtering a Large Dataset - FlatGeobuf, accessed June 11, 2026, <https://flatgeobuf.org/examples/maplibre/large.html>
44. Literate Flatbush: Understanding a fast, elegant RTree implementation. | Kyle Barron, accessed June 11, 2026, <https://kylebarron.dev/blog/literate-flatbush/>
45. RBush — a high-performance JavaScript R-tree-based 2D spatial index for points and rectangles - GitHub, accessed June 11, 2026, <https://github.com/mourner/rbush>
46. kdbush vs rbush vs geokdbush vs flatbush | Spatial Indexing Libraries for 2D and Geospatial Data in JavaScript - npm-compare.com, accessed June 11, 2026, <https://npm-compare.com/flatbush.geokdbush.kdbush.rbush>
47. geojson-rbush CDN by jsDelivr - A CDN for npm and GitHub, accessed June 11, 2026, <https://www.jsdelivr.com/package/npm/geojson-rbush>
48. kdbush - NPM, accessed June 11, 2026, <https://www.npmjs.com/package/kdbush>
49. GitHub - mourner/flatbush: A very fast static spatial index for 2D points and rectangles in JavaScript, accessed June 11, 2026, <https://github.com/mourner/flatbush>
50. Flatbush: A very fast static spatial index for 2D points and rectangles in JS | Hacker News, accessed June 11, 2026, <https://news.ycombinator.com/item?id=18185045>
51. Valhalla vs OSRM | What are the differences? - StackShare, accessed June 11, 2026, <https://stackshare.io/stackups/osrm-vs-valhalla>
52. Routing - GraphHopper Directions API, accessed June 11, 2026, <https://docs.graphhopper.com/openapi/routing>
53. GraphHopper Routing Engine 11.0 Released, accessed June 11, 2026, <https://www.graphhopper.com/blog/2025/10/14/graphhopper-routing-engine-11-0-released/>
54. osrm-vs-valhalla.md - open-source-spec - GitHub, accessed June 11, 2026, <https://github.com/Telenav/open-source-spec/blob/master/osrm/doc/osrm-vs-valhalla.md>
55. GraphHopper in the Browser: TeaVM makes Offline Route Planning possible in JavaScript!, accessed June 11, 2026, <https://www.graphhopper.com/blog/2014/05/04/graphhopper-in-the-browser-teavm-makes-offline-routing-via-openstreetmap-possible-in-javascript/>
56. BRouter - OpenStreetMap Wiki, accessed June 11, 2026,

- <https://wiki.openstreetmap.org/wiki/BRouter>
57. Advanced OSM Routing - BRouter, accessed June 11, 2026,
<https://brouter.de/brouter/>
58. App: Offline routing (BRouter) - Implemented features - Kurviger Forum, accessed June 11, 2026,
<https://forum.kurviger.com/t/app-offline-routing-brouter/649>
59. Tile structure - Valhalla Docs - GitHub Pages, accessed June 11, 2026,
<https://valhalla.github.io/valhalla/tiles/>
60. accessed June 11, 2026,
<https://valhalla.github.io/valhalla/tiles/#:~:text=Tiles%20are%20arranged%20into%20a%20hierarchy%20with%20three%20levels.&text=Highway%20roads%3A%20motorway%2C%20trunk%20and%20primary.&text=Arterial%20roads%3A%20secondary%20and%20tertiary.&text=Local%20roads%3A%20unclassified%2C%20residential%2C%20service%20or%20other>
61. Path algorithm - Valhalla Docs, accessed June 11, 2026,
<https://valhalla.github.io/valhalla/thor/path-algorithm/>
62. jansoft/mbujkanji-valhalla-wasm 0.1.2 on npm - Libraries.io, accessed June 11, 2026, <https://libraries.io/npm/@jansoft%2Fmbujkanji-valhalla-wasm>
63. WebAssembly Based In-Browser Offline Code Execution - VTechWorks - Virginia Tech, accessed June 11, 2026,
<https://vtechworks.lib.vt.edu/items/cca8d1c8-dfb4-48df-8c09-0a2e0dca31d3>
64. OAuth 2.0 for Client-side Web Applications - Google for Developers, accessed June 11, 2026,
<https://developers.google.com/identity/protocols/oauth2/javascript-implicit-flow>
65. Creating an OAuth App in Dropbox - Apideck, accessed June 11, 2026,
<https://www.apideck.com/blog/creating-an-oauth-app-for-dropbox>
66. PKCE: What and Why? - Dropbox Tech Blog, accessed June 11, 2026,
<https://dropbox.tech/developers/pkce--what-and-why->
67. Including Dropbox app key and Google oauth client id in client-side code?, accessed June 11, 2026,
<https://community.remotestorage.io/t/including-dropbox-app-key-and-google-oauth-client-id-in-client-side-code/845>
68. Storage | Capacitor Documentation, accessed June 11, 2026,
<https://capacitorjs.com/docs/guides/storage>
69. iOS iCloud Drive Synchronization Deep Dive - Carlo Zottmann, accessed June 11, 2026,
<https://zottmann.org/2025/09/08/ios-icloud-drive-synchronization-deep.html>
70. [preferences] Sync Preferences over iCloud for iOS · Issue #2308 · ionic-team/capacitor-plugins - GitHub, accessed June 11, 2026,
<https://github.com/ionic-team/capacitor-plugins/issues/2308>
71. PWA iOS Limitations and Safari Support [2026] - MagicBell, accessed June 11, 2026,
<https://www.magicbell.com/blog/pwa-ios-limitations-safari-support-complete-guide>
72. Apple's PWA Limitations Are Deliberate, Not Negligence – A Push to Keep Users in

- the App Store - Reddit, accessed June 11, 2026,
https://www.reddit.com/r/PWA/comments/1n6e22q/apples_pwa_limitations_are_d_eliberate_not/
73. Do Progressive Web Apps Work on iOS? The Complete Guide for 2026 - MobiLoud, accessed June 11, 2026,
<https://www.mobiloud.com/blog/progressive-web-apps-ios>
74. Up to 4GB of memory in WebAssembly - V8.dev, accessed June 11, 2026,
<https://v8.dev/blog/4gb-wasm-memory>
75. Understanding valhalla memory consumption profile · valhalla valhalla · Discussion #4816 · GitHub, accessed June 11, 2026,
<https://github.com/valhalla/valhalla/discussions/4816>
76. Memory Settings for Wasm in WebGL - Unity Discussions, accessed June 11, 2026,
<https://discussions.unity.com/t/memory-settings-for-wasm-in-webgl/730431>
77. Valhalla server memory usage increases linearly. · Issue #5172 - GitHub, accessed June 11, 2026, <https://github.com/valhalla/valhalla/issues/5172>